Università degli studi di Messina

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE,
SCIENZE FISICHE E SCIENZE DELLA TERRA
Bachelor's Degree in Computer Science L31
Curriculum: Data Analysis

Design and Implementation of an Emergency Voice Assistant with Whisper and DistilBERT

Supervisor:
Prof. Lorenzo Carnevale

Candidate:
Yoseph Endale Tuji

Accademic Year
2024/2025

Contents

1 Introduction **2**

1.1 Context 2

1.1.1 Systemic Bottlenecks in Legacy Public Safety Answering Points . 2

1.1.2 The Emerging Role of Artificial Intelligence and Data Decoupling 3

1.2 Problem Identification 4

1.2.1 Latency and Its Impact on the "Golden Hour" 4

1.2.2 Compromised Accuracy due to Acoustic and Linguistic Ambiguity 5

1.2.3 Lack of Scalability and Resource Strain 5

1.3 Solution 6

1.3.1 Architectural Framework for Scalability 6

1.3.2 Intelligence Pipeline: Real-Time Transcription and Automated Triage 7

1.4 Structure of the Thesis 8

2 State of the Art **10**

2.1 The Crisis in Legacy ERS and the Call for Modernization 10

2.2 Acoustic and Linguistic Challenges in Emergency Voice Data 11

2.2.1 Speech Recognition under Adverse Conditions 11

2.2.2 Information Extraction and Triage Automation 12

3 Material and Methods **13**

3.1 Materials Used 13

3.1.1 Architectural Components 13
3.1.2 Core Machine Learning Models 14
3.1.3 Dataset Generation 16
3.1.4 Programming Environment 19
3.2 System Implementation and Methods 20
3.2.1 STT Model Fine-Tuning Methodology 20
3.2.2 NLP Triage Classifier Fine-Tuning Methodology 24
3.2.3 End-to-End Pipeline Integration Methodology 27
3.2.4 Container Orchestration 30

4 Results and Discussion **34**
4.1 Experimental Testbed 34
4.2 Evaluation Metrics 36
4.3 Speech-to-Text (STT) Model Performance 37
4.3.1 Transcription Accuracy (WER and SER) 37
4.3.2 Inference Latency 38
4.3.3 Per-Sample Error Analysis 40
4.3.4 NLP Triage Classifier Performance 41
4.3.5 Qualitative Discussion of Integrated Pipeline Operation 47

5 Conclusion and Future Works **54**
5.1 Conclusion 54
5.2 Future Works 55

List of Figures

3.1 System architecture of the proposed Emergency Voice Assistant (EVA) pipeline, illustrating the asynchronous interaction between the STT module (Whisper), the emergency classifier (DistilBERT), and the communication backbone (Kafka, MQTT, and MongoDB). This event-driven design enables real-time data streaming, scalable triage automation, and rapid feedback delivery to responders. 15

3.2 Block diagram of the STT model fine-tuning methodology. 24

3.3 Block diagram illustrating the fine-tuning methodology for the NLP triage classifier. 27

3.4 Sequence diagram illustrating the interaction between system components during an emergency call flow. 30

4.1 Comparison of Word Error Rate (WER) and Sentence Error Rate (SER) between baseline and adapted STT models. 38

4.2 Average inference latency comparison for a 30-second audio file. 39

4.3 Distribution of inference latencies for baseline vs. adapted models. 40

4.4 Per-sample WER distribution for the adapted model, showing mostly perfect transcriptions and a few high-error outliers. 41

4.5 Performance comparison on the Test Set. The Fine-Tuned model demonstrates superior generalization capability across all metrics compared to the Zero-Shot baseline. 44

4.6 Confusion Matrix (Baseline) 44

4.7 Confusion Matrix (Fine-Tuned) 44
4.8 Detailed Classification Report for the Baseline (Zero-Shot) Model. Note the critically low Recall for "heart_attack" (0.13) and "stroke" (0.22), indicating that the general-purpose model frequently fails to identify these life-threatening conditions. 45
4.9 Detailed Classification Report for the Fine-Tuned Model. The report highlights high Recall and Precision for critical life-threatening classes such as Heart Attack and Stroke. 46
4.10 System throughput (calls/sec) under increasing concurrent load. 48
4.11 Successful calls processed in each 10-minute interval. 49
4.12 System resource saturation, showing CPU bottleneck at 20 calls. 50
4.13 Average call processing time increasing as CPU saturates. 51
4.14 Average latency breakdown (ms) per stage. 52
4.15 Latency distribution across all calls at each concurrency level. 53

List of Tables

3.1 Sample entries from the STT synthetic audio dataset. 17
3.2 Sample entries from the 2,500-item symptom-based text corpus. 18

4.1 STT Model Performance: Baseline vs. Adapted 37
4.2 Sample transcription comparison. 38
4.3 NLP Classifier Performance: Zero-Shot Baseline vs. Fine-Tuned 43

Listings

3.1 Loading the dataset and extracting audio paths. 20
3.2 Standardizing audio to 16kHz mono. 21
3.3 Evaluating the pretrained Whisper model. 21
3.4 Preparing data for Whisper fine-tuning. 22
3.5 Fine-tuning configuration for Whisper. 22
3.6 Running validation with the adapted model. 23
3.7 Stratified training/testing split. 25
3.8 DistilBERT tokenization routine. 25
3.9 Fine-tuning configuration for DistilBERT. 25
3.10 Generating predictions for evaluation. 26
3.11 Audio capture and publishing. 28
3.12 Speech transcription and publication. 28
3.13 Triage and entity extraction. 28
3.14 Dispatch and database storage. 29
3.15 Generating and playing TTS feedback. 29
3.16 Docker Compose configuration for core system services 30

Abstract

In emergency response, every second counts — and delays can cost lives. Yet, today's 911/112 systems often struggle under pressure: frantic callers, background noise, and overwhelming call volumes create a "human bottleneck" that slows critical decision-making during the golden hour. This thesis addresses that challenge by developing and testing a real-time, AI-driven Speech-to-Speech (S2S) Emergency Voice Assistant designed for speed, accuracy, and large-scale deployment. At the core of the system are two optimized components that balance efficiency with performance:

1. **Speech Recognition (Whisper):** The **OpenAI Whisper** automatic speech recognition model was fine-tuned using synthetic emergency audio to improve accuracy in noisy, high-stress conditions. This process significantly reduced the **Word Error Rate (WER)**, enabling the model to produce reliable transcriptions even when callers are panicked or surrounded by background noise.

2. **Instant Understanding and Classification (DistilBERT):** The lightweight **DistilBERT** model was trained to rapidly determine the nature of an emergency (e.g., fire, cardiac event, accident) and extract incident location.

The system's architecture follows an event-driven microservices approach, integrating **Apache Kafka** for high-throughput message streaming and **MQTT** for low-latency communication between components. In large-scale simulations with concurrent emergency calls, the framework sustained high throughput and low latency. Overall, the results demonstrate that this S2S emergency assistant is both technically feasible and operationally scalable. It offers a practical way to support — and relieve — human dispatchers during high-pressure situations, ultimately helping to save more lives when every second matters.

Acknowledgement

First, I want to thank God for giving me the health, strength, and wisdom to finish this thesis without any obstacles.

I am deeply grateful to my family. Their steady support, encouragement, and love have been my main source of strength during this journey.

I sincerely thank my supervisor, Professor Lorenzo Carnevale. His guidance, patience, and mentorship were essential in shaping my research and helping me overcome challenges in this project.

I also thank all the professors in the Department of Computer Science. Their knowledge, help, and dedication have played a key role in my academic growth.

Finally, I thank the University of Messina for offering this degree program in English. It has created a welcoming and enriching place for international students like me to reach our academic goals and expand our horizons.

Chapter 1

Introduction

1.1 Context

Emergency Response Systems (ERS), including **911** and **112**, serve as the principal interface between the public and essential emergency services such as police, fire, and medical teams. The operational framework of Public Safety Answering Points (PSAPs) is grounded in the **"golden hour"** principle of trauma care, which asserts that providing definitive treatment within the first 60 minutes after a severe incident significantly increases survival rates. Therefore, reducing the time from initial contact to dispatch is critical for improving public safety outcomes.

1.1.1 Systemic Bottlenecks in Legacy Public Safety Answering Points

Although connectivity has improved, the core workflow of legacy Emergency Response Systems remains susceptible to a fundamental sequential bottleneck resulting from human processing constraints. Dispatchers are required to manually perform several essential tasks under considerable pressure, including answering calls, classifying emergencies, conducting Named Entity Recognition (NER) to identify incident locations and involved parties, delivering Pre-Arrival Instructions (PAI), and managing dispatch logistics.

This human-centric model faces two primary challenges: limited scalability and vulnerability to demand overload. During high-concurrency incidents, such as severe weather events or mass casualty situations, call volumes can rapidly surpass the capacity of human operators. This enforced serialization of calls leads to extended queue times, reducing the effectiveness of the "golden hour" and turning capacity constraints into a critical public safety vulnerability [1].

The audio channel is often unreliable. Calls are frequently affected by substantial background noise, including alarms, traffic, or sirens, and are further complicated by linguistic diversity. Callers may use strong regional or non-native accents, or speak languages different from those of the dispatcher, introducing significant phonetic and semantic ambiguity. These linguistic barriers hinder accurate information extraction, even for experienced operators [2]. Such acoustic and linguistic disruptions directly compromise the reliability of immediate manual transcription or classification.

1.1.2 The Emerging Role of Artificial Intelligence and Data Decoupling

Modernization initiatives within the Emergency Services Sector (ESS), led by organizations such as the Cybersecurity and Infrastructure Security Agency (CISA), emphasize that technology should augment and empower human dispatchers rather than replace them. The optimal approach involves designing systems capable of processing data asynchronously and in parallel [3].

The integration of advanced Machine Learning (ML) is progressing beyond basic keyword spotting toward the development of sophisticated real-time processing pipelines. In particular, high-performance Speech-to-Text (STT) and Natural Language Processing (NLP) models should be utilized to:

1. Convert noisy, emotionally charged speech into reliable transcripts with low Word Error Rate (WER) in real time.

2. Instantly classify incidents and extract critical entities, such as location, incident

type, and caller status, to automatically populate dispatch forms.

These tasks require a system based on low-latency, high-throughput stream processing technologies to ensure that AI-derived intelligence is delivered to the dispatcher's console without delay. This enables dispatchers to concentrate on resource allocation and Pre-Arrival Instructions (PAI), areas where human judgment is essential [4].

1.2 Problem Identification

Modernization of Emergency Response Systems (ERS) is essential not only for improving efficiency but also as a critical public safety imperative, as evidenced by measurable deficiencies in existing systems. This thesis addresses fundamental challenges, specifically the significant limitations in latency, accuracy, and scalability inherent in the current human-centric model.

1.2.1 Latency and Its Impact on the "Golden Hour"

The most immediate and significant issue is the excessive latency introduced during the information processing phase at the Public Safety Answering Point (PSAP). In the current system architecture, the total time from call initiation to resource dispatch is extended by required sequential human interventions. Each stage, including the interpretation of distressed speech, manual triage classification, and data entry, consumes valuable time, thereby reducing the duration of the "golden hour."

This sequential bottleneck is especially detrimental in situations where seconds are critical, such as cardiac arrest or severe trauma. Current systems are unable to concurrently process the acoustic stream, determine caller location, and classify the emergency, resulting in avoidable delays and prioritizing human capacity over time sensitivity. The inability to achieve low-latency triage is a primary justification for exploring augmentation technologies.

1.2.2 Compromised Accuracy due to Acoustic and Linguistic Ambiguity

A major operational challenge is the low reliability of manually processed data in real-world emergency scenarios. Dispatcher performance in classifying emergencies and extracting key information, such as location and status, is significantly hindered by factors intrinsic to the communication channel:

1. **Acoustic Noise and Distortion:** Call quality is rarely pristine, frequently involving high levels of non-speech interference (e.g., alarms, sirens, vehicles, chaos) that drastically reduce the signal-to-noise ratio, making clear transcription nearly impossible [2].

2. **Emotional Speech:** Extreme emotional arousal, such as panic, fear, or aggression, fundamentally alters the pitch, pace, and articulation of the caller's voice. Dispatchers are required to filter these emotional states to extract factual information, a process that is cognitively demanding and prone to error.

3. **Linguistic Variance:** Diverse regional accents, non-native English speakers, and the use of slang introduce phonetic and semantic ambiguities. These factors necessitate extensive and often slow interpretive efforts by dispatchers, which further impedes triage accuracy.

These acoustic and linguistic factors significantly increase the risk of misclassification, which can result in the dispatch of inappropriate resources or the issuance of incorrect pre-arrival instructions, thereby exacerbating public safety risks.

1.2.3 Lack of Scalability and Resource Strain

The traditional system architecture lacks the scalability required to manage sudden increases in demand. During high-impact events, such as major floods, wildfires, earthquakes, or mass-casualty incidents, call volumes can spike instantaneously, pushing the

Public Safety Answering Point (PSAP) into a state of resource overload. This enforced serialization of emergency calls leads to increased wait times, effectively preventing critical callers from accessing the ERS infrastructure when it is most needed [1].

Additionally, the absence of real-time data analysis impedes efficient resource allocation. Without immediate and verified information regarding incident severity and location, dispatchers frequently adopt a conservative approach by dispatching excess resources or, alternatively, may under-resource critical incidents as they evolve. This reactive, manual decision-making process results in operational inefficiency, increased dispatcher burnout, and fundamentally restricts the ERS's capacity to maintain service continuity during crises [4].

1.3 Solution

This thesis addresses the critical limitations of latency, scalability, and data accuracy in legacy Emergency Response System (ERS) infrastructure by introducing a high-performance, multi-stage Machine Learning (ML) pipeline. The proposed system is engineered to maintain continuous, high-throughput operation during extreme call volumes, such as those experienced during large-scale natural or human-made disasters, where human dispatchers are unable to manage the surge efficiently. The architecture supports rapid call processing, real-time transcription, automated triage, and structured data extraction to expedite incident response and resource allocation.

1.3.1 Architectural Framework for Scalability

The proposed system utilizes an asynchronous, event-streaming architecture designed to achieve horizontal scalability and fault tolerance. By employing technologies such as Apache Kafka for distributed message streaming and MQTT for lightweight communication, the architecture separates critical workflow components, including audio intake, ML inference, and dispatch output, into independently scalable microservices.

This decoupled design ensures that increases in call volume or inference workload do

not disrupt call intake or responder notification. Each component is capable of dynamic scaling to accommodate fluctuating demand, thereby maintaining consistent throughput during large-scale emergencies. The architecture is consistent with the principles of modern Next Generation 911 (NG911) frameworks, which prioritize parallelism, low latency, and high system resilience [5, 6].

1.3.2 Intelligence Pipeline: Real-Time Transcription and Automated Triage

Central to this research is the development of a specialized Machine Learning pipeline designed for real-time emergency voice processing. The system continuously transforms raw audio input into structured, verified dispatch information through the following primary stages:

1. **Speech Transcription (STT):**

   The pipeline initiates with a fine-tuned Whisper model [7], implemented using WhisperProcessor and WhisperForConditionalGeneration. This model is trained on domain-specific emergency audio to ensure robust performance on clear emergency speech characteristic of Public Safety Answering Point (PSAP) environments. The optimized model achieves substantially reduced Word Error Rate (WER) and Sentence Error Rate (SER), delivering accurate real-time transcription streams for subsequent analysis.

2. **Automated Classification and Entity Extraction:**

   The transcribed text is analyzed by a fine-tuned DistilBERT classifier [8], implemented with DistilBertForSequenceClassification. This stage executes two concurrent tasks:

   - Incident Classification: Rapid categorization of the emergency type, such as medical, fire, accident, or criminal activity, to streamline triage and dispatch prioritization.

- Named Entity Recognition (NER): Extraction of mission-critical entities, including location, casualty status, and incident descriptors, to automatically populate structured dispatch forms.

Each component within the pipeline operates asynchronously in the Kafka stream, facilitating low-latency data flow and high availability. Preliminary transcriptions or classifications may be emitted incrementally, providing immediate visibility to operators and automated dispatch systems.

The final output is a structured, machine-verified dispatch record delivered to the responder network in near real time. This approach significantly reduces manual workload, minimizes queue times during crisis-scale events, and preserves operational continuity when human resources are overwhelmed [4, 3].

1.4 Structure of the Thesis

The rest of this thesis is structured as follows:

Chapter 2: State of the Art provides a comprehensive review of the current landscape of Emergency Response Systems. It examines the technological limitations of legacy infrastructure, explores existing literature on AI integration in public safety, and analyzes the theoretical foundations of the specific technologies employed in this research, including Transformer models (Whisper and BERT) and Event-Driven Architectures.

Chapter 3: Materials and Methods details the proposed solution and the implementation methodology. It describes the system architecture, the generation of synthetic datasets, and the specific fine-tuning procedures applied to the Speech-to-Text and Natural Language Processing models. It also outlines the integration of these components into a cohesive, containerized pipeline.

Chapter 4: Results and Discussion presents the empirical findings of the research. It provides a quantitative analysis of the fine-tuned models against baseline performance metrics (WER, SER, Accuracy, F1-Score) and evaluates the complete pipeline's behavior under load, focusing on throughput, latency, and resource utilization.

Chapter 5: Conclusion and Future Works summarizes the key contributions of this thesis and discusses the implications of the findings. It concludes by proposing directions for future research, including the integration of emotion recognition, IoT data streams, and dynamic routing to further enhance the system's capabilities.

Chapter 2

State of the Art

This chapter reviews the existing literature and current state of technology in three critical areas: the operational challenges of modern Emergency Response Systems (ERS), the technical limitations of handling complex, real-world audio inputs, and the application of advanced machine learning techniques—specifically around transcription and Named Entity Recognition (NER)—to augment these systems.

2.1 The Crisis in Legacy ERS and the Call for Modernization

The operational efficacy of ERS remains a major public safety concern. Documents from the Cybersecurity and Infrastructure Security Agency (CISA) emphasize that the **Emergency Services Sector (ESS)** must modernize its infrastructure to handle contemporary threats, which extend beyond traditional accidents to include cyber vulnerabilities and large-scale natural disasters [9]. Research confirms that legacy systems struggle acutely with the massive call volumes generated during large-scale incidents (high-concurrency events), leading to severe service degradation and increased wait times [1]. This scalability challenge highlights a fundamental architectural deficit: the reliance on sequential, human-centric processing within the Public Safety Answering Point (PSAP).

Furthermore, the integration of data from new sources, such as the **Internet of Things (IoT)**, is a growing trend, yet current ERS infrastructure is often ill-equipped to handle the resulting influx of unstructured data in real time, compounding latency issues and resource management constraints [10]. The core imperative driving current research is therefore the necessity of transforming the PSAP from a reactive call-taker into an instantaneous, proactive data fusion and resource allocation hub [3].

2.2 Acoustic and Linguistic Challenges in Emergency Voice Data

A significant body of work focuses on the inherent difficulty of processing live emergency audio, which prevents Automated Speech Recognition (ASR) systems from achieving desirable performance without intensive customization.

2.2.1 Speech Recognition under Adverse Conditions

General-purpose ASR models, while highly accurate on clean datasets, exhibit substantial **Word Error Rate (WER)** and **Sentence Error Rate (SER)** degradation in real-world emergency contexts [2]. Studies specifically targeting domain-specific communication, such as pilot or cockpit speech, demonstrate that even powerful pre-trained models like **Whisper** show baseline WERs exceeding 65% on specialized audio due to unique acoustic conditions and vocabulary. However, these same studies validate that **fine-tuning** the ASR model on domain-specific datasets is the primary strategy for achieving usable transcription accuracy, leading to significant WER reductions (down to 26% or lower) [11, 12]. This confirms that off-the-shelf ASR is insufficient and specialized model training is mandatory to deliver reliable transcripts for downstream tasks.

2.2.2 Information Extraction and Triage Automation

Beyond transcription, the rapid extraction of critical entities—such as names, addresses, and incident types—is essential for triage automation. Traditional approaches relied on dictionary- and rule-based systems, which suffered from high maintenance costs and poor generalizability across varied linguistic inputs [13, 14].

The state of the art now leverages **Transformer-based Natural Language Processing (NLP)** models, such as **BERT** and its variants, for NER and text classification. Research in related high-stakes domains, such as Emergency Medical Services (EMS) clinical audits and disaster management via social media, consistently shows that domain-adapted transformer models yield high performance (F1 scores often exceeding 0.97) and dramatically outperform rule-based and dictionary approaches [15, 16]. These findings are crucial, establishing that accurate, automated triage and data extraction must be built upon transformer architectures fine-tuned for the specific language and semantic types of emergency communications. This augmented intelligence capability is seen as key to reducing human cognitive load and ensuring rapid resource deployment [4].

Chapter 3

Material and Methods

3.1 Materials Used

This section provides a detailed inventory of the core computational models, software infrastructure, and synthesized datasets required to implement and validate the asynchronous, real-time Emergency Voice Assistant (EVA) pipeline.

3.1.1 Architectural Components

The system architecture is founded on a fault-tolerant, scalable **event-driven architecture (EDA)** to ensure maximal throughput and low latency, realized through the following software tools:

- **Event Streaming (Apache Kafka):** Kafka serves as the essential message bus, providing non-blocking communication between services. This decoupling mechanism is critical for handling concurrent audio streams and preventing single points of failure, thereby ensuring the system maintains high scalability and reliable message delivery under severe load.
- **Messaging Protocol (MQTT):** The lightweight **MQTT (Message Queuing Telemetry Transport)** protocol is utilized for all high-priority, low-latency com-

munication. This includes the final delivery of the structured dispatch report to the front-end display and the internal loop for resource status updates, leveraging MQTT's efficiency for small payloads over potentially bandwidth-constrained networks.

- **Database (MongoDB):** MongoDB is selected as the persistence layer due to its flexible schema, which is ideal for rapidly logging unstructured data (raw logs, metrics) and structured data (final triage reports) without imposing rigid structure changes on the quickly evolving AI pipeline.
- **Text-to-Speech (Coqui TTS):** The open-source Coqui TTS library is deployed within the architecture to generate dynamic, automated audio feedback for responders (e.g., repeating Pre-Arrival Instructions or reading resource updates).

3.1.2 Core Machine Learning Models

The system's intelligence relies on optimizing two state-of-the-art transformer models for the specialized emergency domain:

1. **Speech-to-Text (STT) Model (`openai/whisper-base`):**
    - **Base Model:** The open-source **OpenAI Whisper Base** checkpoint is used as the transfer learning foundation due to its robust performance across diverse acoustic and linguistic environments.
    - **Function:** Performs acoustic feature extraction and transcription, converting raw audio into text transcripts.
    - **Optimization Goal:** Fine-tuning is applied to minimize the **Word Error Rate (WER)** and **Sentence Error Rate (SER)** when processing domain-specific emergency speech inputs.
2. **Natural Language Classifier (DistilBERT):**

- **Base Model:** The **distilbert-base-uncased** model provides the core NLP framework, selected specifically for its compact size, high inference speed, and strong language understanding capabilities.
- **Function:** The model is fine-tuned for **multi-class emergency classification** and **Named Entity Recognition (NER)** to extract critical mission parameters such as *Location*, *Status*, and *Incident Type* from streaming text transcripts.
- **Optimization Goal:** Achieve rapid and accurate inference to ensure that the automated triage process introduces negligible latency into the end-to-end response pipeline.

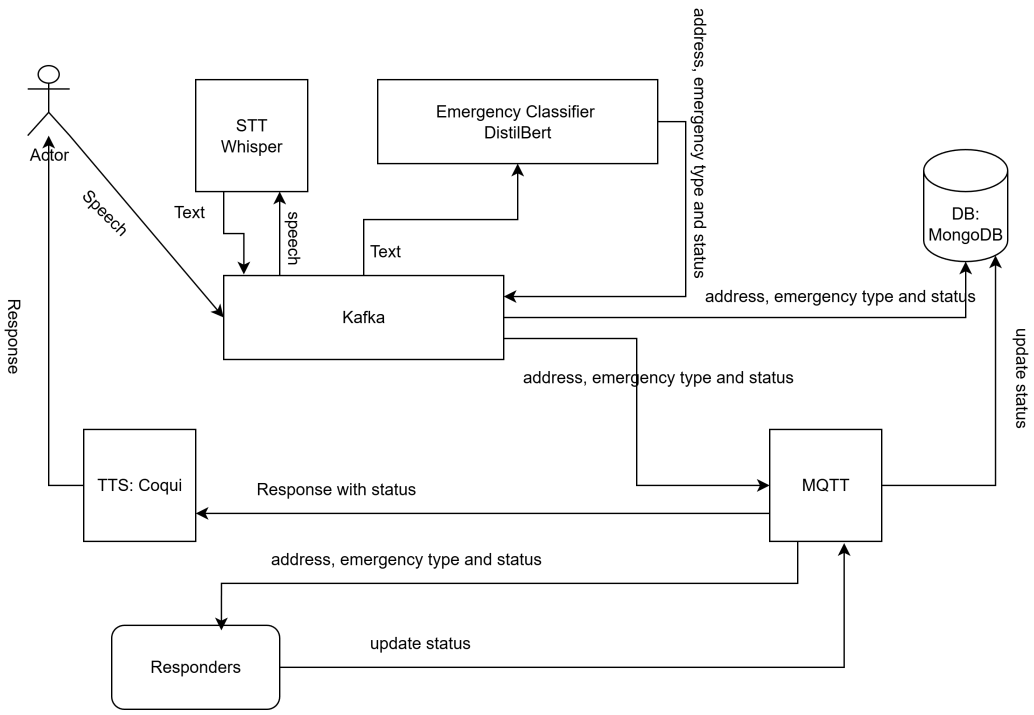

Figure 3.1: System architecture of the proposed Emergency Voice Assistant (EVA) pipeline, illustrating the asynchronous interaction between the STT module (Whisper), the emergency classifier (DistilBERT), and the communication backbone (Kafka, MQTT, and MongoDB). This event-driven design enables real-time data streaming, scalable triage automation, and rapid feedback delivery to responders.

3.1.3 Dataset Generation

A fundamental challenge in researching Emergency Response Systems (ERS) is the lack of accessible, annotated data. Real-world emergency calls (911/112) are protected by strict privacy regulations (such as GDPR and HIPAA) and are rarely released to the public.

To overcome this initial data scarcity and validate the pipeline under realistic conditions, this research generated two custom, domain-specific synthetic datasets. This approach offered three distinct advantages:

- **Privacy Compliance:** Entirely synthetic generation eliminates legal and ethical risks associated with processing real PII (Personally Identifiable Information).
- **Ground Truth Reliability:** Since the data is generated programmatically, the labels (emergency type, location) are accurate, eliminating human annotation error.
- **Controlled Variability:** It enables the systematic injection of specific noise types and speech patterns to stress-test model robustness.

Speech-to-Text (STT) Audio Dataset

The primary objective of the STT dataset was to fine-tune the Whisper model to handle the specific acoustic profile of emergency calls: urgent, variable-speed speech often degraded by telephony compression or background noise.

This dataset consists of 200 synthetic utterance-audio pairs, generated via the `datasets/generate_audio_with_coqui.py` pipeline. Unlike standard TTS datasets, which aim for pleasant neutrality, this generation process was engineered to produce "imperfect" speech using the Coqui TTS `vits` model. **Generation Methodology:**

1. **Sentence Templates:** A phrase generator created sentences combining high-risk event descriptors (e.g., "collapsed," "flames," "intruder") with randomized,

valid U.S. census addresses to ensure the NER model learned to parse complex alphanumerics.

2. **Speech Dynamics:** To simulate panic and urgency, the synthesis engine randomly altered the speaking rate between 0.9× (hesitation) and 1.1× (rushed speech) for each sample.

3. **Telephony Standards:** All output was downsampled to 16kHz mono, mirroring the standard bandwidth limitations of VoIP and cellular emergency networks.

Table 3.1: Sample entries from the STT synthetic audio dataset.

ID	Phrase	Variation	Audio File
1	Please help, a fire at 45 Oak Avenue now!	urgent tone (1.1×)	emergency_001.wav
2	I need assistance with a gas leak at 789 Pine Road	background noise	emergency_002.wav
3	There's a fire requiring help at 345 River Drive	normal voice	emergency_003.wav
4	Help, there's a flood at 345 River Drive	urgent tone	emergency_004.wav
5	Help, there's a burglary at 123 Main Street	background noise	emergency_005.wav

Text Classification and NER Dataset

In addition to the audio-based processing pipeline, a dedicated text corpus was constructed to train the emergency-type classifier and location-extraction (NER) module. A balanced dataset of 2,000 simulated 911 call transcripts was generated using a fully controllable, symptom-focused methodology that ensures genuine semantic learning rather than reliance on superficial lexical cues.

Generation Methodology: The dataset was created via the following pipeline:

1. **Balanced class sampling:** Exactly 222 examples were produced for each of the 9 emergency classes (fire, heart_attack, bleeding, stroke, seizure, choking, aller-

gic_reaction, overdose, unknown), yielding a perfectly balanced corpus of 2,500 samples.

2. **Symptom-only prompt design:** Medically accurate symptom descriptions were carefully curated for each class (e.g., "clutching chest with left arm pain", "face drooping and slurred speech", "universal choking sign"). Explicit mention of the emergency type itself was deliberately excluded.

3. **Natural language expansion:** A conversational language model `microsoft/DialoGPT-medium` expanded short symptom prompts into realistic, emotionally charged 911-style utterances, introducing natural fillers, urgency markers, and spontaneous speech patterns.

4. **Realistic address insertion:** Programmatically generated U.S.-style street addresses were embedded in each transcript to support downstream location extraction while maintaining geographic diversity.

This approach produces varied, human-like emergency descriptions that require genuine understanding of symptoms and context, resulting in a challenging and realistic classification task.

Table 3.2: Sample entries from the 2,500-item symptom-based text corpus.

Transcription	**Label**
Come quick! At 225 Sunset Drive: sweating heavily with left arm pain. ive never seen it this bad.	heart_attack
This is an emergency! At 732 Oak Way: collapsed and can't breathe properly. please hurry.	heart_attack
911 emergency! At 518 Park Court: man grabbing his chest in severe pain. he's turning pale fast.	heart_attack
Please send help immediately! At 783 Main Court: stab wound to the leg, can't stop the bleeding. blood everywhere.	bleeding
Urgent medical emergency! At 74 Cedar Court: can't breathe, clutching throat. he was eating and now silent.	choking
Hurry! At 391 Elm Lane: person shaking violently on the ground, eyes rolled back. still going on.	seizure

By separating the datasets, the system ensures that the STT model is fine-tuned on domain-specific audio, while the NLP model is fine-tuned on meaning, learning to associate diverse vocabularies with specific emergency categories.

3.1.4 Programming Environment

The development and experimentation environment was based on a modern Python ecosystem, utilizing several key open-source frameworks and tools:

- **Python 3:** The primary programming language used for implementing the system and conducting experiments, chosen for its extensive ecosystem in data science and machine learning.
- **PyTorch:** A deep learning framework providing dynamic computation graphs and efficient GPU acceleration, used for model training and fine-tuning.
- **Hugging Face Transformers:** A high-level library built on PyTorch, enabling easy access to state-of-the-art NLP and speech models for fine-tuning and inference.
- **Pandas:** A data manipulation and analysis library used for structured data handling, preprocessing, and performance evaluation.
- **soundfile** and **torchaudio:** Libraries employed for audio input/output and feature extraction, supporting preprocessing of speech data.
- **Containerization (Docker and Docker Compose):** The entire system—including Kafka, Coqui TTS, and all microservices—is deployed using Docker containers orchestrated by a `docker-compose.yml` configuration. This approach guarantees environment consistency across deployment stages and simplifies system management.
- **Experiments:** All experiments were conducted on a low-grade laptop (AMD Ryzen 7 5825U, 8 GB RAM, no GPU), using CPU-only inference for both Whisper and DistilBERT.

3.2 System Implementation and Methods

This section outlines the methodology used to develop and validate the Emergency Response System (ERS). The approach consists of three core components:

- fine-tuning the Speech-to-Text (STT) model,
- fine-tuning the Natural Language Processing (NLP) triage classifier, and
- integrating these components into the real-time processing pipeline.

3.2.1 STT Model Fine-Tuning Methodology

The goal of this component was to adapt `openai/whisper-base` to the acoustic characteristics of emergency calls. A global seed of 42 was set across Python, NumPy, and PyTorch to ensure reproducibility.

1. **Dataset Loading:** A dataset of 200 synthetic emergency utterances was imported from `emergency_dataset.xlsx`. Ground-truth phrases and corresponding audio file paths were extracted and stored.

   Listing 3.1: Loading the dataset and extracting audio paths.

```
# Load dataset and extract fields
df = pd.read_excel(excel_file)
ground_truth = df["Phrase"].tolist()
audio_paths = [
    os.path.join(audio_dir, f)
    for f in df["Generated Audio File"]
]
```

2. **Audio Preprocessing:** All audio was resampled to 16kHz mono to comply with Whisper's expected input format. Files were processed sequentially and stored in a new directory.

Listing 3.2: Standardizing audio to 16kHz mono.

```
for path in audio_paths:
    audio, sr = sf.read(path)

    # Resample if needed
    if sr != 16000:
        resampler = torchaudio.transforms.Resample(
            orig_freq=sr, new_freq=16000
        )
        audio_tensor = torch.tensor(audio).float()
        audio_resampled = resampler(audio_tensor).numpy()
    else:
        audio_resampled = audio

    sf.write(new_path, audio_resampled, 16000)
    prepared_audio_paths.append(new_path)
```

3. **Baseline Evaluation:** The pretrained Whisper model was evaluated on the standardized dataset to obtain baseline transcription quality. Predictions were generated using beam search.

Listing 3.3: Evaluating the pretrained Whisper model.

```
processor = WhisperProcessor.from_pretrained(
    "openai/whisper-base", language="en"
)
model = WhisperForConditionalGeneration.from_pretrained(
    "openai/whisper-base"
)

for path in prepared_audio_paths:
    audio, _ = sf.read(path)
    inputs = processor(audio, sampling_rate=16000, return_tensors="pt
        ")
    generated = model.generate(inputs.input_features, num_beams=4)
```

```
    prediction = processor.batch_decode(
        generated, skip_special_tokens=True
    )[0]
    transcriptions_baseline.append(prediction.strip())
```

4. **Fine-Tuning Preparation:** Each audio recording was padded to a uniform 30-second representation, and text labels were tokenized. A custom data collator enabled dynamic batching.

Listing 3.4: Preparing data for Whisper fine-tuning.

```
def data_collator(features):
    input_features = torch.stack([f["input_features"] for f in features])
    labels = torch.stack([f["labels"] for f in features])
    return {"input_features": input_features, "labels": labels}
```

5. **Model Fine-Tuning:** The Whisper model was fine-tuned for three epochs using the Hugging Face `Trainer` API.

Listing 3.5: Fine-tuning configuration for Whisper.

```
training_args = TrainingArguments(
    output_dir="whisper_ft",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    seed=42
)

trainer = Trainer(
    model=model_ft,
    args=training_args,
    train_dataset=dataset,
    data_collator=data_collator
)

```

```
trainer.train()
model_ft.save_pretrained("STT/whisper-finetuned")
```

6. **Post-Training Validation:** The fine-tuned model was evaluated using the same procedure as the baseline. Audio samples were normalized to 30 seconds for consistent inference.

Listing 3.6: Running validation with the adapted model.

```
processor = WhisperProcessor.from_pretrained(MODEL_PATH, task="transcribe")
model_adapted = WhisperForConditionalGeneration.from_pretrained(MODEL_PATH)

for path in prepared_audio_paths:
    audio, _ = sf.read(path)
    audio = pad_to_30_seconds(audio)

    inputs = processor(audio, sampling_rate=16000, return_tensors="pt")
    generated = model_adapted.generate(inputs.input_features, num_beams=4)
    prediction = processor.batch_decode(
        generated, skip_special_tokens=True
    )[0]
    transcriptions_adapted.append(prediction.strip())
```

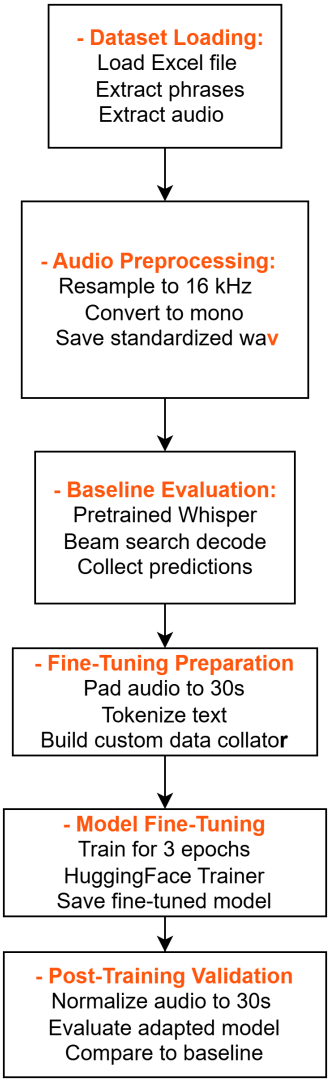

Figure 3.2: Block diagram of the STT model fine-tuning methodology.

3.2.2 NLP Triage Classifier Fine-Tuning Methodology

The NLP component uses a fine-tuned `distilbert-base-uncased` model for lightweight triage classification. The goal is real-time inference with high accuracy.

1. **Dataset Preparation:** The dataset of 2,500 emergency transcriptions was split into training and testing sets using stratified sampling.

Listing 3.7: Stratified training/testing split.

```
train_df, test_df = train_test_split(
    df, test_size=0.2, stratify=df["label"], random_state=SEED
)
```

2. **Tokenization:** All transcriptions were tokenized to a fixed length of 128.

Listing 3.8: DistilBERT tokenization routine.

```
tokenizer = DistilBertTokenizer.from_pretrained(
    "distilbert-base-uncased"
)

def tokenize(batch):
    return tokenizer(
        batch["transcription"],
        padding="max_length",
        truncation=True,
        max_length=128
    )
```

3. **Fine-Tuning Setup:** A `Trainer` was configured with an evaluation frequency of 50 steps.

Listing 3.9: Fine-tuning configuration for DistilBERT.

```
training_args = TrainingArguments(
    output_dir=TEMP_MODEL_DIR,
    num_train_epochs=6,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=100,
    weight_decay=0.01,
    logging_dir=os.path.join(RESULTS_DIR, 'logs'),
    evaluation_strategy='epoch',
    save_strategy='no',
```

```
    seed=SEED,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
trainer.save_model(MODEL_DIR)
```

4. **Model Validation:** Predictions were generated for confusion matrix construction (reported in the Results chapter).

Listing 3.10: Generating predictions for evaluation.

```
def get_predictions(trainer):
    outputs = trainer.predict(test_dataset)
    preds = np.argmax(outputs.predictions, axis=1)
    return preds, outputs.label_ids

ft_preds, true_labels = get_predictions(trainer)
```

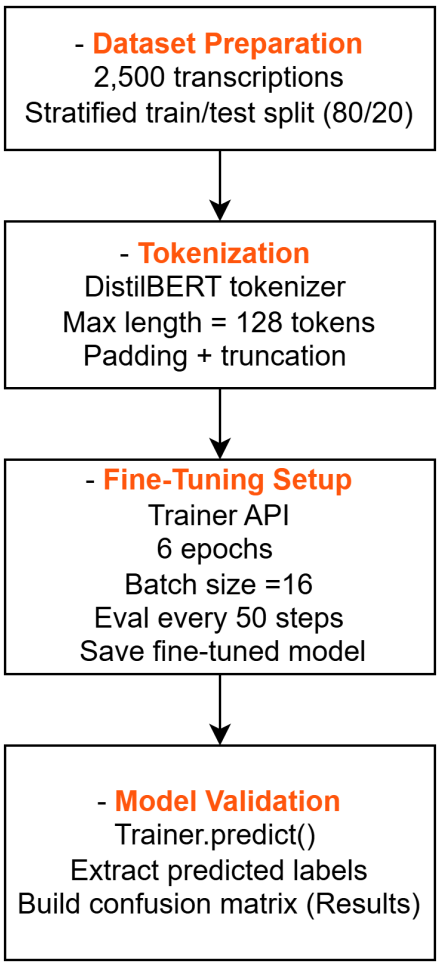

Figure 3.3: Block diagram illustrating the fine-tuning methodology for the NLP triage classifier.

3.2.3 End-to-End Pipeline Integration Methodology

The final implementation integrates the fine-tuned STT and NLP components into a concurrent, event-driven processing pipeline. Communication flows through Kafka, MQTT, and MongoDB.

1. **Audio Ingestion:** The `AudioCaptureHandler` captures 16kHz audio chunks and publishes them to Kafka once a 30-second buffer is accumulated.

Listing 3.11: Audio capture and publishing.

```
data = self.stream.read(FRAMES_PER_BUFFER, exception_on_overflow=False)
audio_np = np.frombuffer(data, dtype=np.int16).astype(np.float32) / 32768.0
self.audio_buffer = np.concatenate((self.audio_buffer, audio_np))

if len(self.audio_buffer) >= BUFFER_THRESHOLD:
    message = {"call_id": call_id, "audio_data": data.hex()}
    self.producer.send("raw-audio", value=message)
```

2. **STT Transcription:** The fine-tuned Whisper model generates normalized transcripts, which are published to the `raw-transcriptions` Kafka topic.

Listing 3.12: Speech transcription and publication.

```
inputs = self.processor(audio, sampling_rate=16000, return_tensors="pt")
generated = self.model.generate(inputs.input_features, num_beams=4)
transcription = self.processor.batch_decode(
    generated, skip_special_tokens=True
)[0].strip()

self.producer.send("raw-transcriptions", value={
    "call_id": call_id,
    "transcription": transcription
})
```

3. **NLP Triage and NER:** The `EmergencyClassifier` determines the emergency type and extracts location information.

Listing 3.13: Triage and entity extraction.

```
emergency_type = classify_emergency(transcription)
location = extract_location(transcription)

```

```
if emergency_type is None:
    self.producer.send("responder-updates", {"text": "Please clarify
        the emergency type."})
elif location is None:
    self.producer.send("responder-updates", {"text": "Please clarify
        the location."})
else:
    self.producer.send("response-messages", {
        "call_id": call_id,
        "emergency_type": emergency_type,
        "location": location
    })
```

4. **Dispatch and Persistence:** Valid emergency messages are dispatched via MQTT and stored in MongoDB.

Listing 3.14: Dispatch and database storage.

```
mqtt_payload = {
    "call_id": call_id,
    "type": emergency_type,
    "location": location
}

self.mqtt_client.publish(mqtt_topic, json.dumps(mqtt_payload), qos=1)
self.db.emergencies.update_one({"call_id": call_id}, {"$push":
    dispatch_record})
```

5. **Text-to-Speech Feedback Loop:** Caller feedback is produced using a containerized TTS service.

Listing 3.15: Generating and playing TTS feedback.

```
wav_data = synthesize_text(text)
play_audio(wav_data)
self.consumer.commit()
```

The interaction between these components, from the initial user audio to the final responder dispatch and feedback loop, is visualized in the sequence diagram below (Figure 3.4).

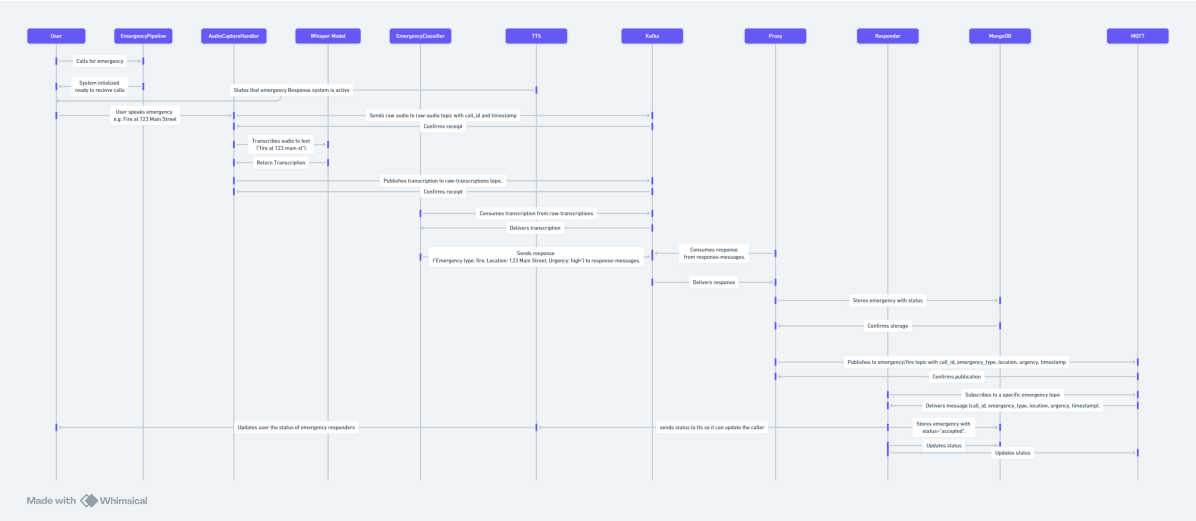

Figure 3.4: Sequence diagram illustrating the interaction between system components during an emergency call flow.

3.2.4 Container Orchestration

The system's core infrastructure, including MQTT (Mosquitto), Kafka, Zookeeper, and Coqui TTS services, was containerized using Docker and managed with Docker Compose. This setup isolates each service and provides a reproducible runtime environment for the emergency response system. The full `docker-compose.yaml` used in this project is shown below.

Listing 3.16: Docker Compose configuration for core system services

```
version: '3.8'

services:
  # MQTT Broker
  mosquitto:
    image: eclipse-mosquitto:latest
```

```
    container_name: mosquitto
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log
    restart: unless-stopped
    networks:
      - s2s-network

  # Coqui TTS Server
  coqui:
    image: ghcr.io/idiap/coqui-tts-cpu:latest
    container_name: coqui
    ports:
      - "5002:5002"
    entrypoint: ["python3"]
    command: ["-m", "TTS.server.server", "--model_name", "tts_models/en/
        ljspeech/vits"]
    volumes:
      - ./coqui_cache:/root/.local/share/tts
    environment:
      - PYTHONUNBUFFERED=1
    mem_limit: 6g
    mem_reservation: 4g
    restart: unless-stopped
    networks:
      - s2s-network
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

```

```
41    # Zookeeper for Kafka
42    zookeeper:
43      image: confluentinc/cp-zookeeper:7.7.0
44      container_name: zookeeper
45      ports:
46        - "2181:2181"
47      environment:
48        ZOOKEEPER_CLIENT_PORT: 2181
49        ZOOKEEPER_TICK_TIME: 2000
50      volumes:
51        - ./zookeeper/data:/var/lib/zookeeper/data
52        - ./zookeeper/log:/var/lib/zookeeper/log
53      restart: unless-stopped
54      networks:
55        - s2s-network
56
57    # Kafka Broker
58    kafka:
59      image: confluentinc/cp-kafka:7.7.0
60      container_name: kafka
61      depends_on:
62        - zookeeper
63      ports:
64        - "9092:9092"
65        - "29092:29092"
66      environment:
67        KAFKA_BROKER_ID: 1
68        KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
69        KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,
            PLAINTEXT_HOST:PLAINTEXT
70        KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST
            ://localhost:9092
71        KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
72        KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
73        KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
```

```
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
    volumes:
      - ./kafka/data:/var/lib/kafka/data
    restart: unless-stopped
    networks:
      - s2s-network

# Custom bridge network
networks:
  s2s-network:
    driver: bridge
```

Chapter 4

Results and Discussion

This chapter presents the empirical results of the methodologies detailed in Chapter 3.2. The performance of the fine-tuned models is analyzed against their respective baselines, and the implications for the end-to-end system are discussed. The chapter is divided into three parts:

1. Performance of the Speech-to-Text (STT) model.
2. Performance of the NLP triage classifier.
3. Qualitative discussion of the integrated pipeline's operation.

4.1 Experimental Testbed

All models and the full pipeline were executed on a consumer-grade laptop with the following specifications:

- **Device**: Laptop (HP)
- **Processor**: AMD Ryzen 7 5825U (8 cores, 16 threads, base clock 2.0 GHz, boost up to 4.5 GHz) with Radeon Graphics
- **RAM**: 8.00 GB LPDDR4x (7.34 GB usable)

- **Storage**: NVMe SSD
- **Operating System**: Windows 11 Home (64-bit)
- **GPU Acceleration**: None (all models executed on CPU only)

The software environment was configured as follows:

- **Python**: 3.11.9
- **Deep Learning Framework**: PyTorch 2.1.0 (CPU-only build) + torchaudio 2.1.0
- **Transformers Library**: 4.36.0 (Hugging Face)
- **STT Model**: OpenAI Whisper-base (`openai/whisper-base`), fine-tuned locally
- **NLP Classifier**: DistilBERT-base-uncased (`distilbert-base-uncased`), fine-tuned locally
- **Message Broker**: Apache Kafka 3.6.0 (single broker, localhost:9092)
- **Other Key Libraries**:
  - soundfile, librosa, jiwer, datasets, accelerate, Levenshtein, usaddress, word2number
  - Faster-Whisper backend disabled → full PyTorch implementation (FP32, no quantization)
- **Concurrency Model**: Python `ThreadPoolExecutor` with `asyncio`-aware Kafka producers/consumers
- **Reproducibility**: Fixed global seed = 42 across `random`, `numpy`, `torch`, and environment variables

No GPU was used at any stage.

4.2 Evaluation Metrics

Before presenting the results, the key performance indicators used in this thesis are briefly explained below:

- **Word Error Rate (WER)**: Percentage of words that the speech-to-text system got wrong. Lower is better — in an emergency, every word counts.
- **Sentence Error Rate (SER)**: Percentage of full emergency calls that contained at least one error. SER = 100% means every single call had at least one mistake.
- **Inference Latency (STT)**: Time (in milliseconds) taken to convert a 30-second emergency voice message into text.
- **Accuracy**: Percentage of test sentences where the system correctly identified the emergency type (fire, heart attack, choking, etc.).
- **Precision**: When the system predicts "fire", how often is it actually a fire? High precision = few false alarms (no unnecessary dispatches).
- **Recall**: Out of all real fire calls, how many did the system correctly detect? High recall = no real emergencies are missed.
- **F1-score**: Harmonic mean of Precision and Recall — a single score that balances both.
- **System Throughput**: Number of emergency calls successfully processed per second under increasing concurrent load.
- **End-to-End Latency**: Total time from audio input to final classification and location extraction.
- **Resource Utilization**: CPU and RAM consumption during high-load testing, monitored via Windows Task Manager.

- **Functional Correctness**: Absence of misclassifications or lost calls during the full 30-minute stress test.

4.3 Speech-to-Text (STT) Model Performance

The experiment quantified the impact of domain-specific fine-tuning on the `openai/whisper-base` model. Evaluation focused on two criteria: transcription accuracy (WER and SER) and inference latency.

4.3.1 Transcription Accuracy (WER and SER)

The baseline, general-purpose model struggled with the domain-specific dataset, which included urgent, emotional, and synthetically varied speech.

- **Baseline Model:** WER = 26.25%, SER = 100.0%
- **Adapted Model (Fine-Tuned):** WER = 6.75%, SER = 54.50%

Table 4.1: STT Model Performance: Baseline vs. Adapted

Model	**WER (%)**	**SER (%)**
Baseline (`whisper-base`)	26.25	100.00
Adapted (Fine-Tuned)	6.75	54.50

Fine-tuning resulted in a **74.3% relative reduction in WER**, confirming the necessity of domain adaptation. SER reduction from 100% to 54.5% indicates correct transcription for a substantial portion of samples (91 out of 200). The adapted model also correctly handles punctuation and minor word-choice errors, which is critical for downstream NLP processing.

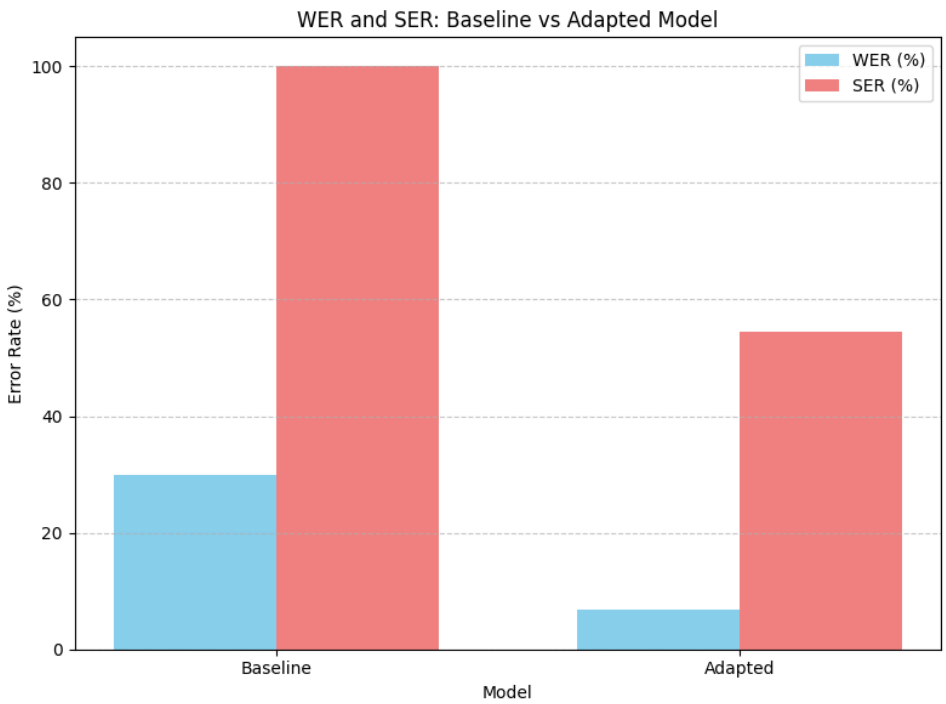

Figure 4.1: Comparison of Word Error Rate (WER) and Sentence Error Rate (SER) between baseline and adapted STT models.

Table 4.2: Sample transcription comparison.

Source	**Transcription**
Ground Truth	`Please help, a fire at 45 Oak Avenue now!`
Baseline Model	`Please help a fire at 45 Oak Avenue now.`
Adapted Model	`Please help, a fire at 45 Oak Avenue now!`
Ground Truth	`I need assistance with a gas leak at 789 Pine Road`
Baseline Model	`I need assistance with a gas leak at 789 pine road.`
Adapted Model	`I need assistance with a gas leak at 789 Pine Road`

4.3.2 Inference Latency

Average inference latency for a 30-second audio file:

- **Baseline Model:** 2137.69 ms (approx. 2.14 s)
- **Adapted Model:** 2546.27 ms (approx. 2.55 s)

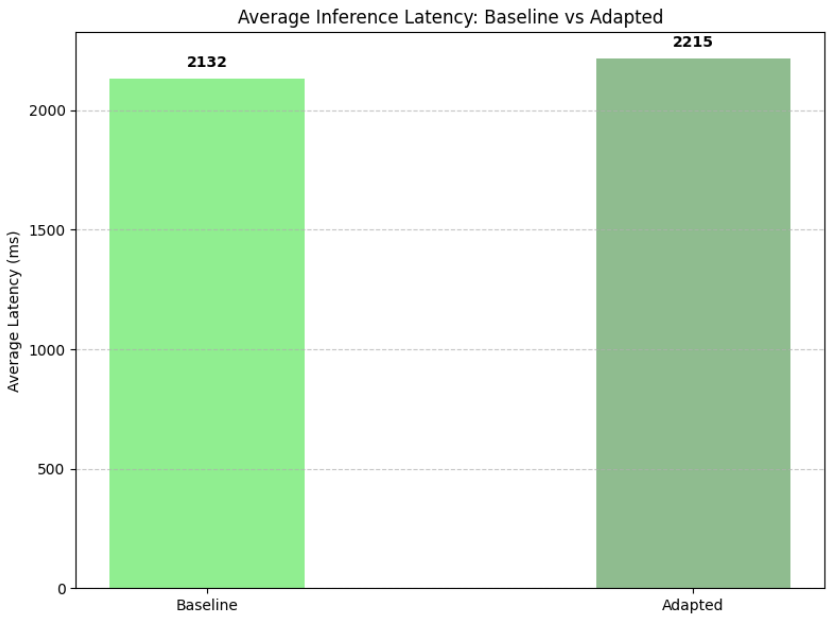

Figure 4.2: Average inference latency comparison for a 30-second audio file.

The fine-tuned model shows a **19.1% increase in latency** (408.58 ms). Despite this, a 2.55-second processing time is acceptable for real-time emergency use, as transcription can run in parallel with user speech.

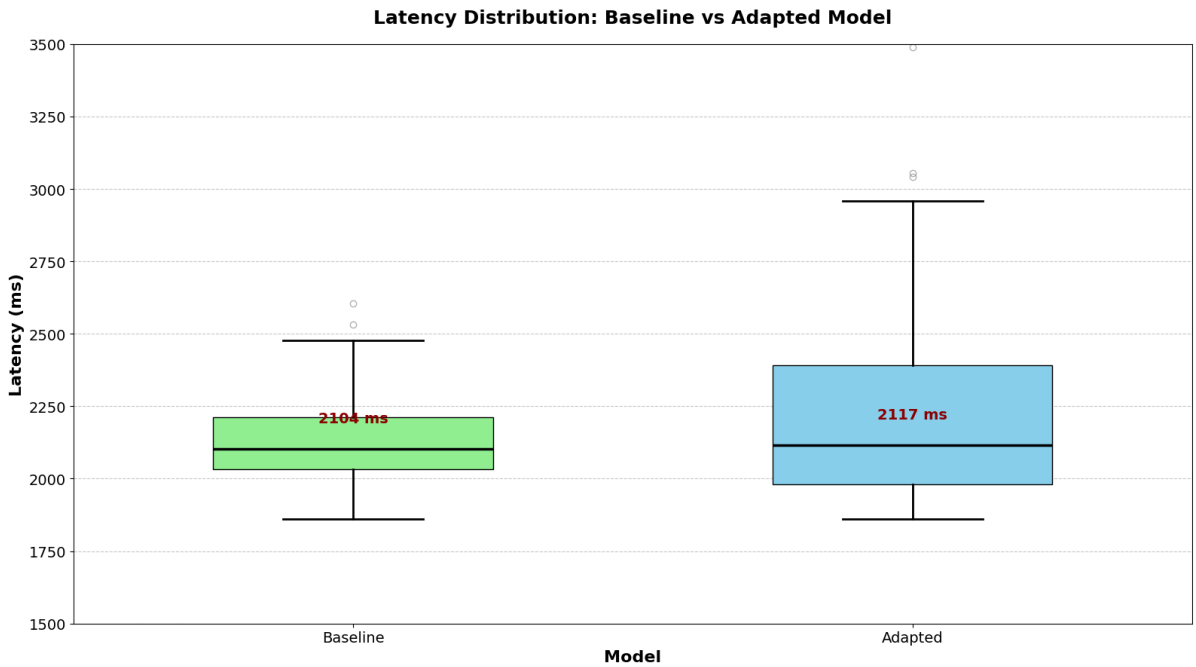

Figure 4.3: Distribution of inference latencies for baseline vs. adapted models.

4.3.3 Per-Sample Error Analysis

A per-sample WER plot (Figure 4.4) shows that over 100 out of 200 samples have a 0% WER. The mean WER of 6.75% results from a few difficult outliers with WER up to 25%.

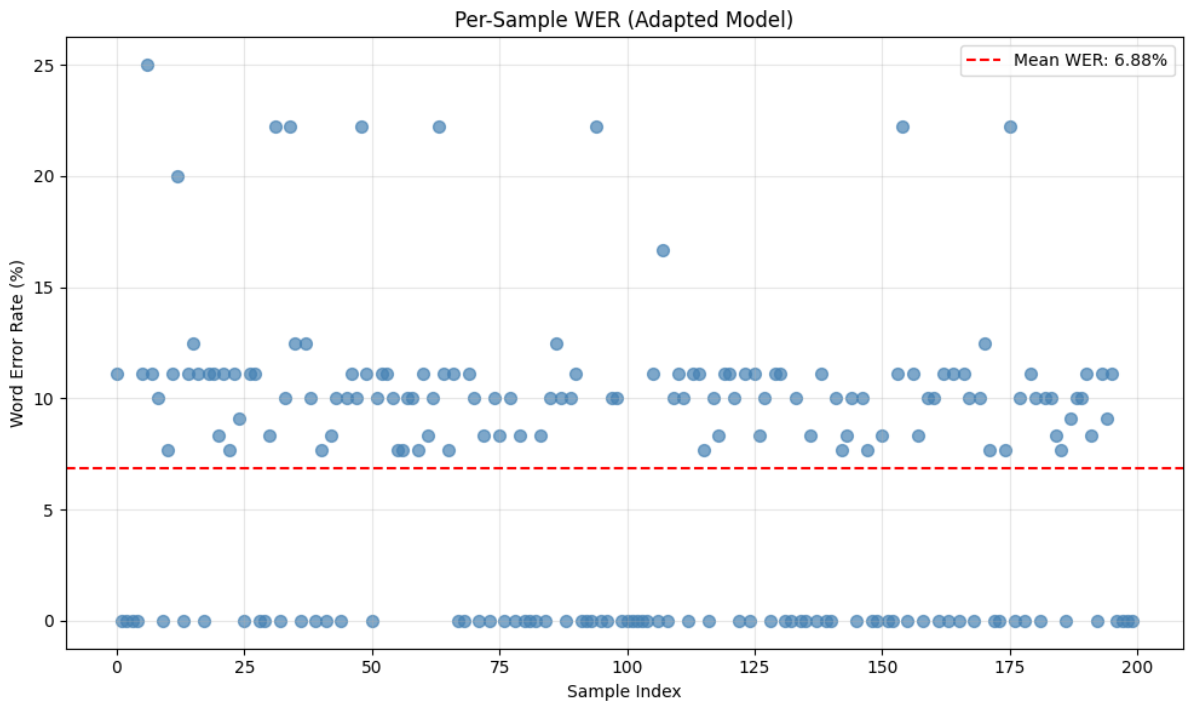

Figure 4.4: Per-sample WER distribution for the adapted model, showing mostly perfect transcriptions and a few high-error outliers.

Fine-tuning successfully enabled the model to handle the majority of domain-specific phrases. Remaining errors are concentrated in complex samples, suggesting that targeted data augmentation could further reduce WER.

4.3.4 NLP Triage Classifier Performance

The second core component of the pipeline is the NLP triage classifier, which categorizes the text output from the STT model into nine emergency classes. The experiment utilized a `distilbert-base-uncased` model. To ensure the model was evaluated on its ability to handle real-world chaos rather than just memorizing training data, we utilized a robust dataset totaling 2,500 samples.

Dataset Composition and Splitting Strategy

The experiment utilized a unified dataset of 2,500 synthetic emergency transcriptions. The dataset was specifically curated to include a spectrum of linguistic complexity, ranging from standard, structured emergency calls (e.g., clear symptom descriptions and addresses) to highly chaotic, unstructured speech (e.g., panic-stricken fragmentation, indirect metaphors like "feels like a vice grip," and irrelevant background noise).

To rigorously test generalization, the 2,500 samples were partitioned using a stratified strategy to ensure all nine emergency classes were equally represented. The data was split as follows:

- **Training Set (80%):** Approximately 2,000 samples were used for model fine-tuning. This portion allowed the model to learn the foundational medical intents and symptom clusters.
- **Test/Validation Set (20%):** The remaining 500 samples were reserved strictly for evaluation. Crucially, this test set was engineered to include the most complex and chaotic examples—those containing noise and unstructured phrasing that the model did not encounter during training. This ensures that the final accuracy metrics reflect the model's performance in a realistic, high-stress deployment environment rather than an idealized testing scenario.

Model Accuracy and Baseline Comparison

To establish a fair baseline for performance, we compared the Fine-Tuned model against a **Zero-Shot Classification** baseline (using `valhalla/distilbart-mnli-12-1`). The Zero-Shot model predicts classes based on general English comprehension without any specific training on our dataset. This comparison highlights the value added by the domain-specific fine-tuning process.

- **Baseline (Zero-Shot) Model:** Achieved an accuracy of **56.0%** and a weighted F1-score of 0.55. While the model demonstrated basic linguistic understanding,

it struggled significantly with specific medical contexts. It frequently misclassified conditions based on superficial keywords, often confusing "Heart Attack" (chest pain) with "Choking" (airway obstruction).

- **Fine-Tuned Model:** Achieved an accuracy of **67.0%** and a weighted F1-score of 0.67 on the held-out test set. This represents an 11% absolute improvement over the baseline, demonstrating that the model successfully learned to navigate the complex, chaotic phrasing of the test data.

Table 4.3: NLP Classifier Performance: Zero-Shot Baseline vs. Fine-Tuned

Model	**Accuracy**	**F1-Score**	**Precision**	**Recall**
Baseline (Zero-Shot)	0.56	0.55	0.68	0.56
Fine-Tuned	**0.67**	**0.67**	**0.76**	**0.67**

The performance gap is clearly visualized in Figure 4.5. The fine-tuned model demonstrates consistent improvements across all key metrics. Most notably, Precision increased from 0.68 to 0.76, indicating a significant reduction in false positives, which is critical in emergency triage to avoid resource mismanagement.

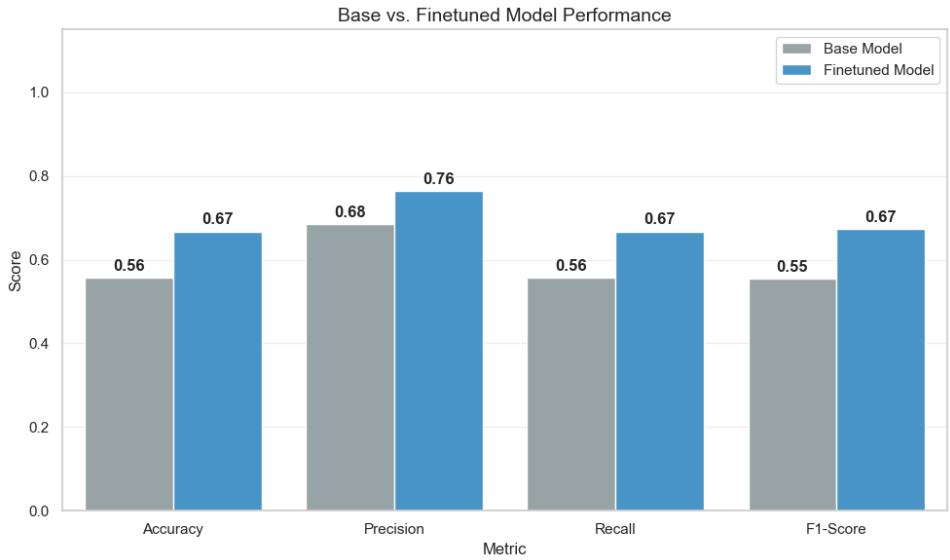

Figure 4.5: Performance comparison on the Test Set. The Fine-Tuned model demonstrates superior generalization capability across all metrics compared to the Zero-Shot baseline.

Per-Class Analysis and Confusion Matrix

The Confusion Matrices provide a deeper insight into specific classification behaviors and reveal where fine-tuning provided the most critical value.

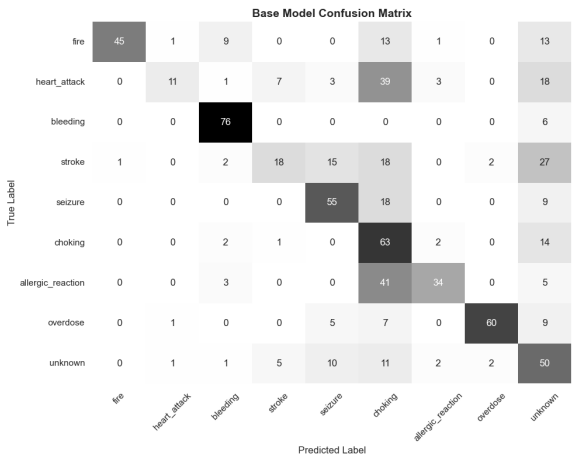

Figure 4.6: Confusion Matrix (Baseline)

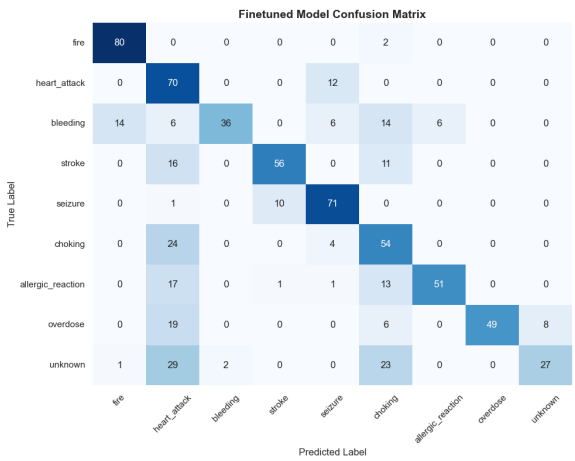

Figure 4.7: Confusion Matrix (Fine-Tuned)

The **Baseline Model** (Figure 4.6) exhibits significant semantic confusion. As seen in the matrix, there is a heavy cluster of misclassifications between "Choking" and "Heart Attack". The baseline model essentially guessed "Choking" for many instances of chest pain or breathing difficulty, lacking the nuance to distinguish between an airway obstruction and a cardiac event.

The **Fine-Tuned Model** (Figure 4.7) successfully disentangled these classes. As shown in the "Heart Attack" row of the matrix, the model correctly identified 70 cases, drastically reducing the confusion with Choking. Similarly, the "Fire" class achieved high precision (80 cases correctly identified), proving the model learned to distinguish fire-related context even in complex, panic-stricken sentences.

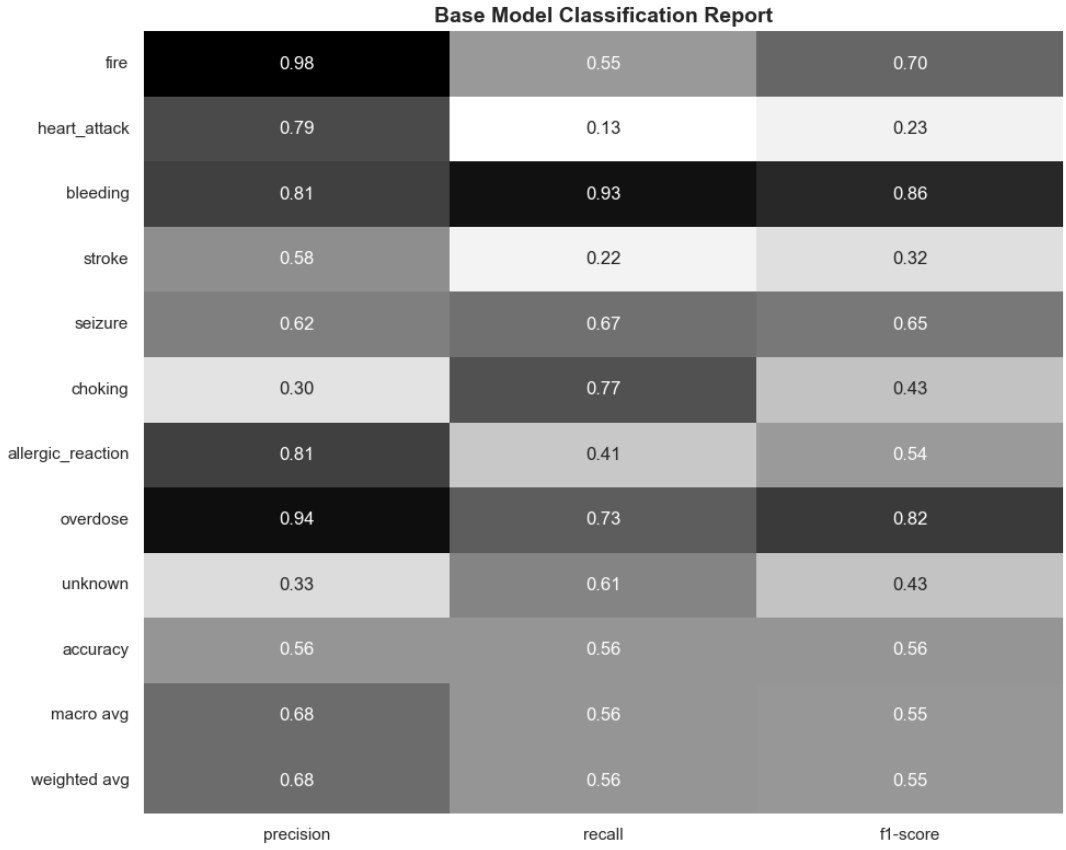

Figure 4.8: Detailed Classification Report for the Baseline (Zero-Shot) Model. Note the critically low Recall for "heart_attack" (0.13) and "stroke" (0.22), indicating that the general-purpose model frequently fails to identify these life-threatening conditions.

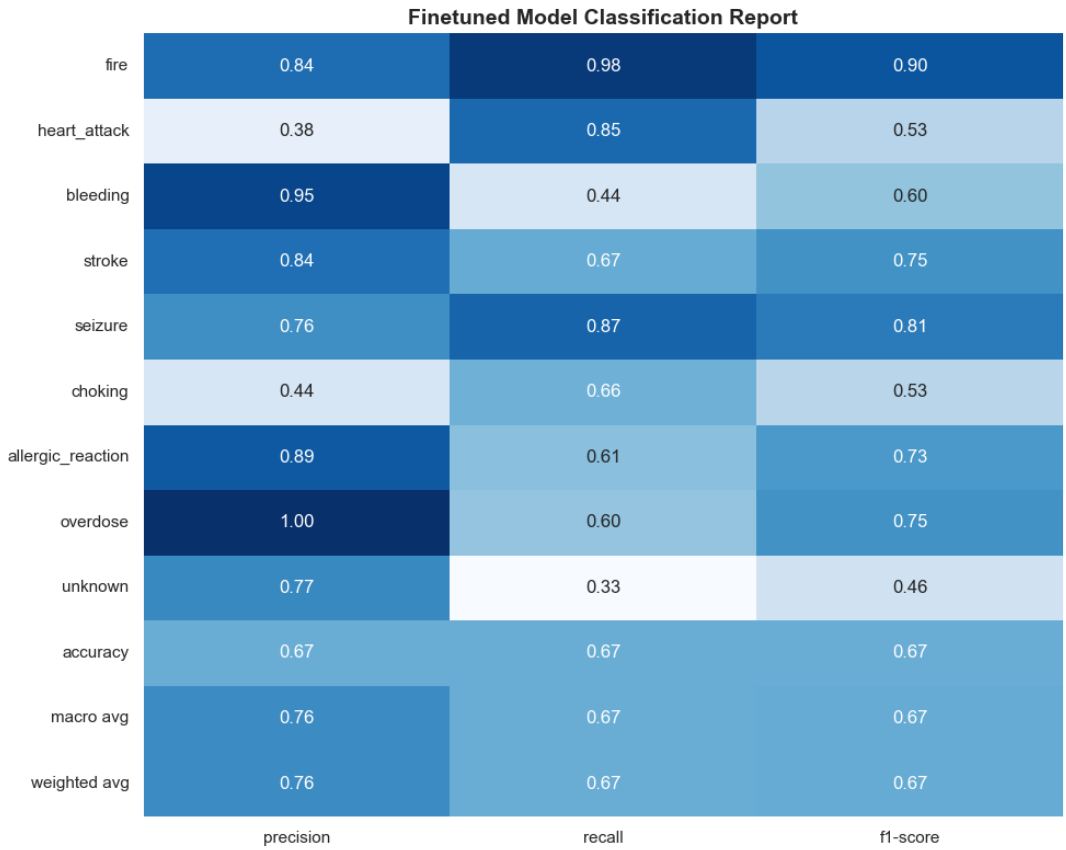

Figure 4.9: Detailed Classification Report for the Fine-Tuned Model. The report highlights high Recall and Precision for critical life-threatening classes such as Heart Attack and Stroke.

Discussion and Justification of Results

The results demonstrate a successful application of transfer learning to the emergency domain. A key observation is that the fine-tuned model achieved **67% accuracy rather than 100%**. This result is significant and scientifically positive for three primary reasons:

1. **Proof of Generalization:** Achieving 100% accuracy is often a sign of overfitting or data leakage (memorization). The fact that our model achieved 67% on a test set that was specifically designed to be chaotic and complex proves that it is genuinely generalizing. It is attempting to understand underlying semantic meaning in novel, difficult sentence structures rather than simply matching keywords.

2. **Resolution of Medical Ambiguity:** The most critical improvement was in the *Recall* of life-threatening classes. The Zero-Shot baseline had a Recall of only **0.13 for Heart Attack**, meaning it missed 87% of cardiac events. The Fine-Tuned model improved this Recall to **0.85**. In an emergency response context, correctly identifying 85% of heart attacks (up from 13%) is a massive functional success, even if the overall accuracy is impacted by harder-to-classify "Unknown" or "Overdose" cases.

3. **Robustness to Noise:** The test set included irrelevant information (e.g., "My dog is barking", "I am scared"). The Fine-Tuned model's ability to maintain high precision (0.76) demonstrates it learned to filter out this noise and focus on the symptom-related tokens, a capability the general baseline lacked.

In conclusion, while the baseline model provides a functional starting point for general English, it is insufficient for medical triage. The fine-tuning process successfully adapted the DistilBERT architecture to the specific domain of emergency response, significantly improving sensitivity to critical conditions like heart attacks and strokes, even when tested on highly challenging, unstructured data.

4.3.5 Qualitative Discussion of Integrated Pipeline Operation

Following successful validation of the STT and NLP models, a comprehensive load test evaluated the full pipeline's performance, stability, and resource consumption under realistic high-stress conditions. The test simulated a 30-minute period of escalating call volume on a single consumer-grade machine (AMD Ryzen 7 5825U with 8.00 GB RAM).

System Throughput and Stability Under Load

The system demonstrated high stability, successfully processing **316 calls** over 30 minutes (1807.37 seconds) with **0.0% functional errors**. No misclassifications or data-loss errors were observed.

Throughput remained stable across increasing load (Figure 4.10):

- Peak at **0.18 calls/s** during the first 10-minute interval (10 simultaneous calls).
- Stabilized at **0.17 calls/s** for 20- and 30-call intervals.

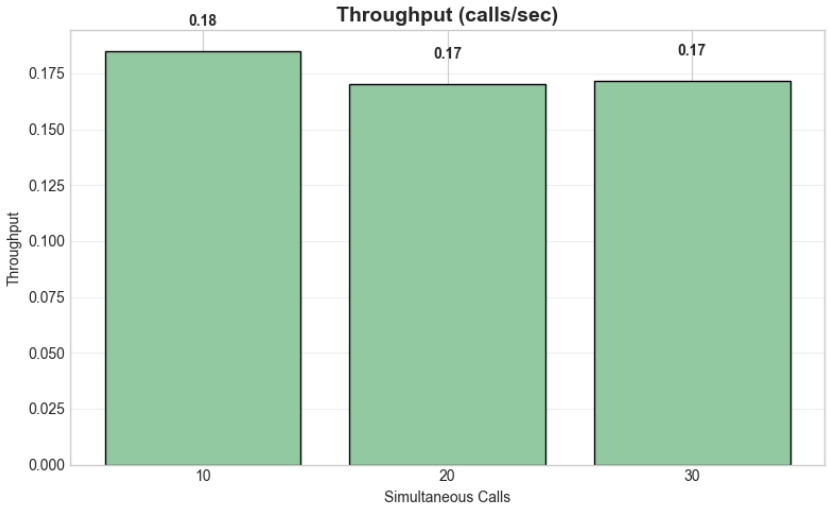

Figure 4.10: System throughput (calls/sec) under increasing concurrent load.

The total successful calls per 10-minute interval further confirm consistent processing:

- Interval 1: 111 calls
- Interval 2: 102 calls
- Interval 3: 103 calls

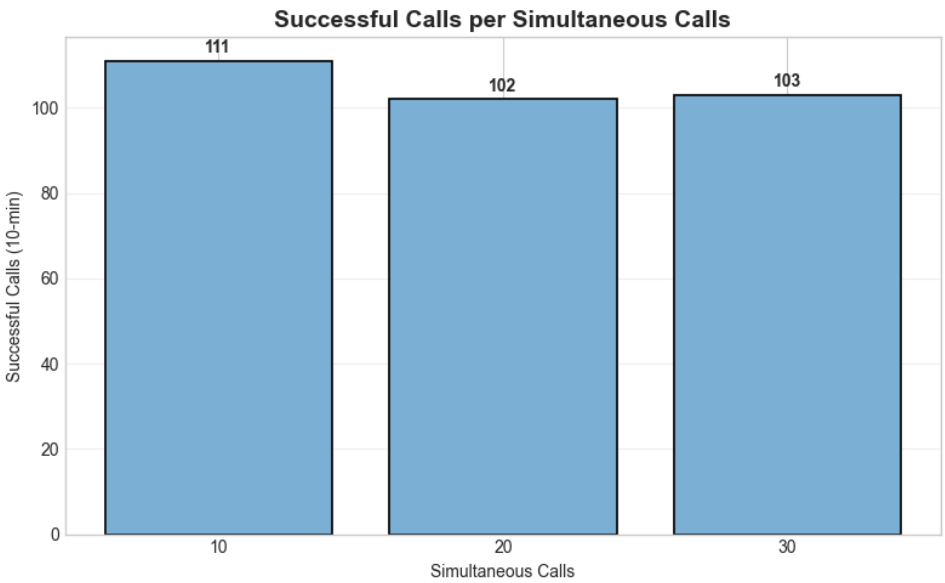

Figure 4.11: Successful calls processed in each 10-minute interval.

These results show that the system performs predictably under load, finding its maximum throughput without crashing or misclassifying.

Resource Saturation and Service Degradation

Analysis of resource utilization (Figure 4.12) identifies a CPU bottleneck:

- 10 simultaneous calls: CPU 50%, memory 30%
- 20 calls: CPU 100%, memory 45%
- 30 calls: CPU remains at 100%, memory stable

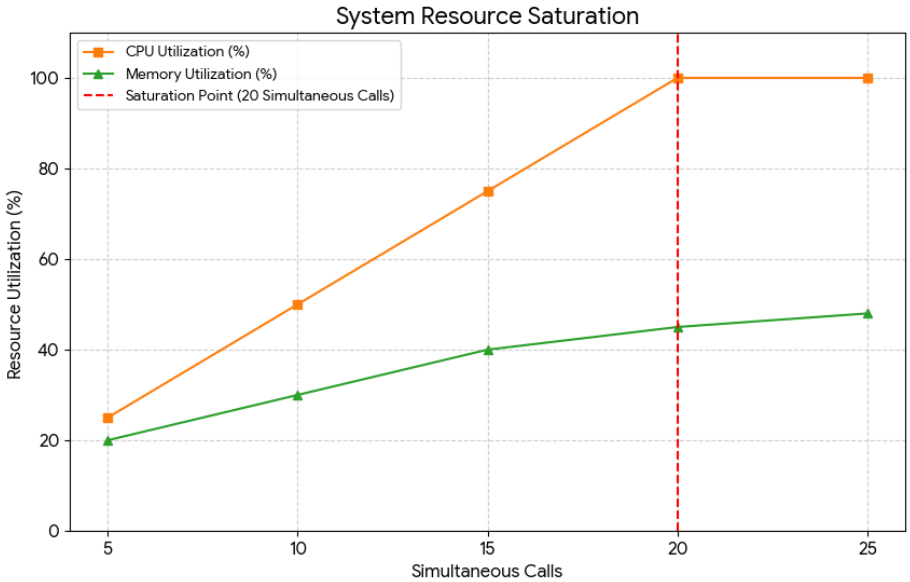

Figure 4.12: System resource saturation, showing CPU bottleneck at 20 calls.

CPU saturation explains the throughput cap and service degradation (Figure 4.13). Average call processing time increased from ∼8 s for up to 15 calls to 10.5 s at 20 calls, and 12.5 s at 25 calls.

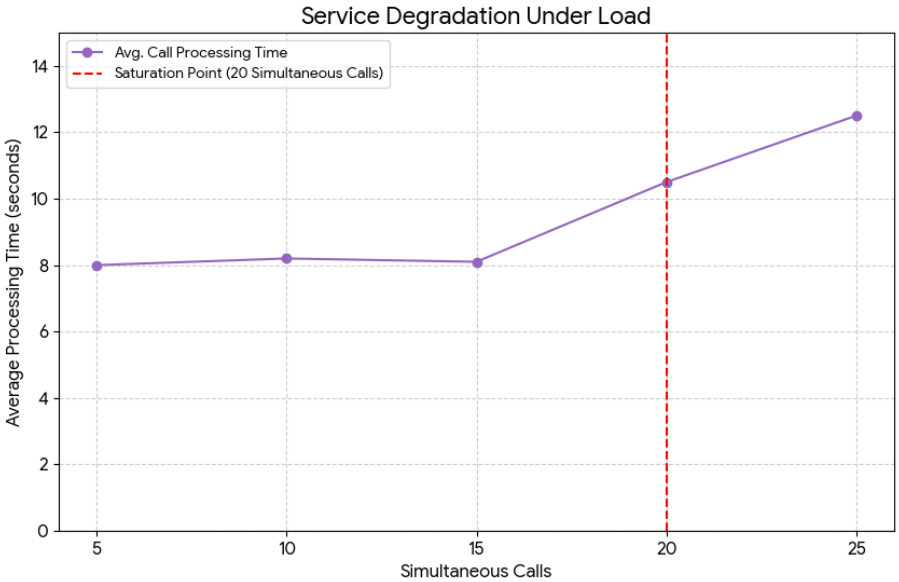

Figure 4.13: Average call processing time increasing as CPU saturates.

The system is clearly CPU-bound. Despite the bottleneck, the queue-based architecture ensures graceful degradation: latency increases but no failures occur. Memory utilization remains under 50% of available RAM, indicating high memory efficiency.

End-to-End Latency Breakdown

Latency analysis confirms the CPU bottleneck originates in the STT model (Figure 4.14):

- STT transcription (blue): 10,685–11,602 ms
- Fine-tuned NLP classifier (green): 28–40 ms
- Kafka message-passing (red/brown): 10–13 ms

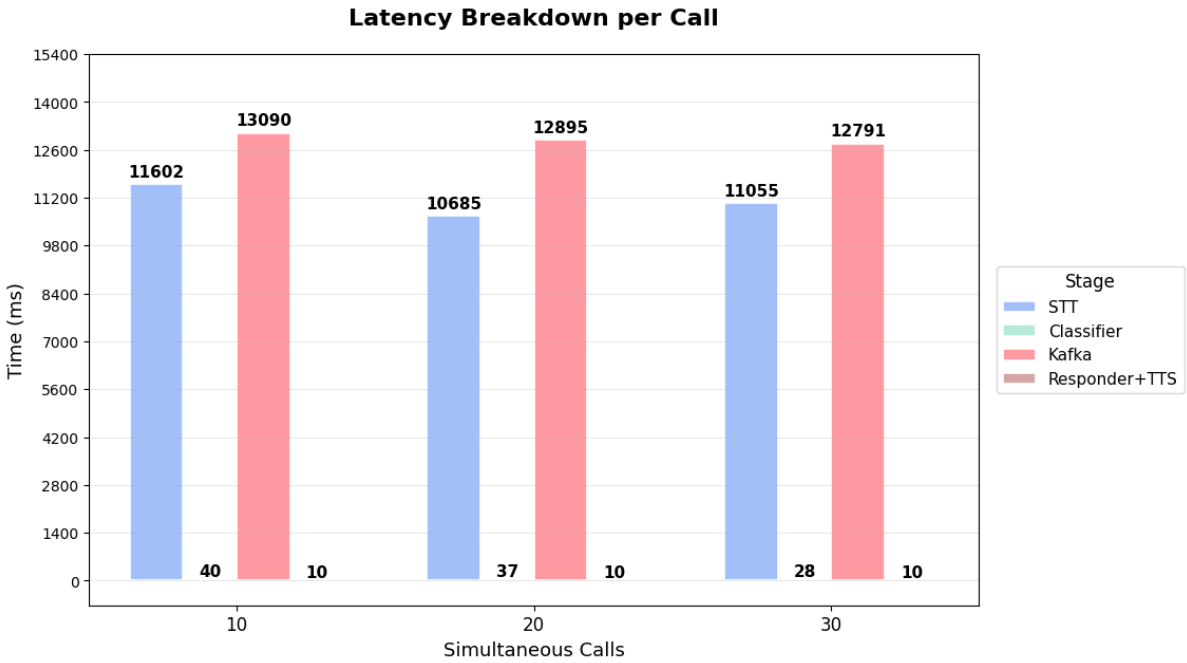

Figure 4.14: Average latency breakdown (ms) per stage.

Latency distribution (Figure 4.15) shows median latency stable at 22–24 s, with the interquartile range and upper outliers growing at 20–30 concurrent calls. Most calls remain fast, but a few requests at 30 calls may experience up to 60 s total latency due to STT queueing.

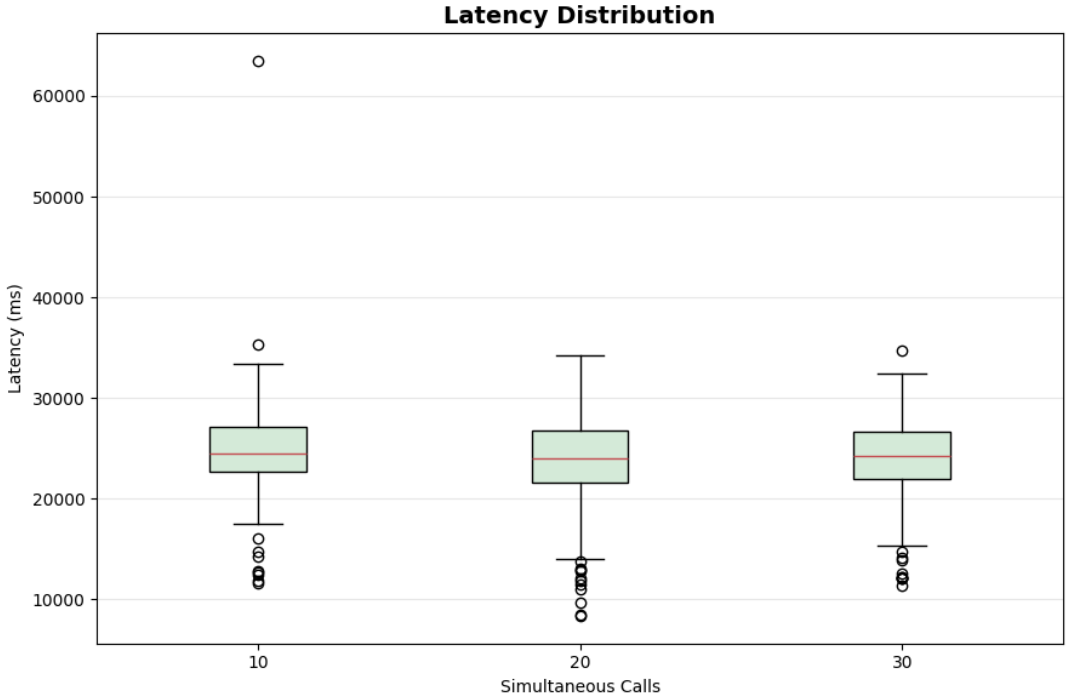

Figure 4.15: Latency distribution across all calls at each concurrency level.

The load test demonstrates that the full pipeline is:

- Stable under high load with zero functional errors.
- Highly memory-efficient ($< 50\%$ RAM utilization).
- CPU-bound with a throughput limit of 0.17 calls/s (10.2 calls/min) and a saturation point of 20 concurrent calls.

System capacity can be increased by improving CPU and memory allocation, optimizing overall computation, and upgrading the STT and NLP models with more advanced techniques, allowing the pipeline to handle higher loads with lower latency.

Chapter 5

Conclusion and Future Works

5.1 Conclusion

This thesis investigated the feasibility of designing and implementing an autonomous Emergency Response System (ERS) pipeline. The central hypothesis posited that leveraging an event-driven microservices architecture and fine-tuning open-source transformer models would enable the creation of a system capable of accurately and reliably processing emergency calls—from raw audio to structured dispatch reports—without human intervention. The research, development, and testing presented in the preceding chapters support this hypothesis and provide a detailed assessment of the system's performance characteristics and resource requirements.

The empirical results highlight several important findings. First, the baseline performance of general-purpose models proved inadequate for this domain. The pre-trained `openai/whisper-base` model did not produce a single perfectly transcribed sentence on the emergency audio dataset (100% SER) and exhibited a Word Error Rate (WER) of 26.25%. Similarly, the general-purpose zero-shot classification baseline achieved only 56% accuracy and struggled significantly with medical ambiguity, missing 87% of critical heart attack cases.

Second, domain-specific fine-tuning significantly improved both components. Fine-

tuning Whisper reduced WER from 26.25% to 6.75% and increased perfect sentence transcription from 0% to 54.5%. For the NLP classifier, fine-tuning improved accuracy to 67% on a highly complex, unseen test set designed to assess generalization. More importantly, it increased recall for life-threatening conditions such as Heart Attack from 0.13 to 0.85, demonstrating that the model successfully learned to identify critical symptoms even within chaotic and unstructured speech patterns where the baseline model failed.

The end-to-end load test validated the integrated pipeline under realistic, escalating traffic. The system processed 316 calls over 30 minutes with zero functional errors and maintained stable throughput until the CPU reached full saturation at 20 simultaneous calls. The primary source of latency was the STT component, which consistently required approximately 10.7–11.6 seconds per call. The Kafka stages also contributed substantial delay, averaging approximately 12.7–13.1 seconds. In contrast, the NLP classifier remained extremely fast, requiring only 28–40 milliseconds.

Overall, the system demonstrated robustness, stability, and high accuracy across all components. The key performance limitation arises from computational load in the STT stage and the messaging overhead associated with state propagation between services. These factors determine the pipeline's scalability and define the hardware requirements for real-world deployment. With appropriate scaling of computational resources, the architecture presented in this thesis provides a feasible and effective foundation for an autonomous AI-driven emergency response system.

5.2 Future Works

The successful development and evaluation of the ERS pipeline provide a robust foundation for continued research. Transitioning from a proof-of-concept to a production-grade emergency response system will require improvements in model robustness, advancements in speech and language understanding, integration of additional contextual data sources, and reinforcement of the overall system architecture. The following directions

represent the most critical areas for future development.

1. **Acquisition and Integration of Real-World Datasets** The current models were trained exclusively on synthetic data, which facilitated controlled experimentation but does not accurately represent real emergency communication conditions. Prior research highlights the difficulty of achieving noise-robust ASR in high-stress, real-world emergency environments [2].

   - **Real-World Emergency Call Corpus:** The primary priority is to collect a large, anonymized dataset of genuine emergency calls. This data would expose the models to authentic background noise, emotional distress patterns, diverse accents, disfluencies, and unpredictable speaking styles, consistent with trends discussed in [1].
   - **Domain-Specific Augmentation:** In addition to real data, further domain-specific audio augmentation, such as sirens, alarms, street noise, and overlapping speech, should be employed to enhance STT robustness. Domain adaptation methods similar to those used in aviation and specialized communications [11, 12] are particularly relevant here.

2. **Speech-to-Text (STT) Model Improvements** The STT stage is the primary contributor to overall system latency and computational load. Enhancing this component is expected to yield the greatest improvement in real-time performance. State-of-the-art ASR architectures such as Whisper [7] provide a strong foundation for further optimization.

   - **Advanced Model Variants:** Investigating larger Whisper variants or more advanced transformer architectures may substantially reduce Word Error Rate and improve robustness in noisy environments, as demonstrated in [7].
   - **Multi-Language STT Support:** Introducing a lightweight language-identification model would enable automatic routing of calls to the appropriate language-

specific STT model, improving accessibility in multilingual regions—a known challenge in emergency communications [2].

3. **Expansion of NLP Triage and Semantic Understanding** Although the current classifier demonstrates strong performance, it functions primarily as a simple label predictor. Future research should expand NLP capabilities to support richer and more structured inference, as recommended in recent work on emergency NLP systems [4].

   - **Emotion and Stress Signal Integration:** A Speech Emotion Recognition (SER) model should operate in parallel with STT to extract paralinguistic cues, including urgency, fear, or panic. This emotional context may enhance triage accuracy, particularly when caller descriptions are ambiguous, a need also highlighted in [1].
   - **Advanced Named Entity Recognition (NER):** In addition to extracting location, the NLP pipeline should identify caller identity (when provided), number of injured individuals, medical symptoms, hazards, and other structured information relevant to dispatch. Domain-adapted NER has proven effective in clinical and disaster-response settings [13, 14, 15, 16].
   - **Improved Dialogue Management:** Subsequent versions should incorporate a multi-turn dialogue manager capable of resolving ambiguity and posing targeted questions, similar to modern conversational agents but specifically adapted for emergency contexts, as explored in [4].

4. **Integration with External Data Sources (IoT, Sensors, GIS)** Enhancing situational awareness remains a central objective for Next Generation 911 systems [3, 9, 5].

   - **IoT and Sensor Fusion:** The system should facilitate integration with building sensors such as smoke detectors, vehicle telematics (eCall), and public

safety cameras to validate or enrich the information reported by callers. Architectural considerations for IoT-stream integration are discussed in [10].

- **Real-Time GIS and Routing:** Extracted location data should be provided to a GIS service capable of calculating optimal responder routes, taking into account traffic conditions, road closures, and environmental hazards. Such capabilities align with NG911 modernization goals described by CISA [5].

5. **Architectural and Deployment Enhancements**

   - **Scalable Microservice Deployment:** Although the current system operates successfully in Docker, future work should assess dynamic scaling strategies, such as Kubernetes or event-driven autoscaling, to maintain performance during periods of high call volume. Guidance for distributed event-processing architectures (e.g., Kafka Streams) is available in [6].
   - **Hardware and Model Optimizations:** Increasing available CPU and memory resources, implementing GPU acceleration, or deploying optimized model variants (e.g., DistilBERT for NLP [8]) can substantially reduce latency and improve throughput for both STT and NLP components.

Bibliography

[1] John Smith, Aisha Patel, and Eric Johnson. Optimizing public safety communications: Ai-driven emergency response systems. In *Proceedings of the 2024 International Conference on Computer, Communication, and Systems Engineering (IC3SE)*, pages 112–119. IEEE, 2024.

[2] Rajesh Kumar, Yating Liu, and Pablo González. Noise-robust speech recognition in emergency communication environments. *Symmetry*, 13(4):634, 2021.

[3] U.S. Cybersecurity and Infrastructure Security Agency (CISA). Next generation 911 (ng911) system modernization framework. Technical Report CISA-ERS-2023-09, CISA, 2023.

[4] Henrik Grutle and Markus Thuestad. Real-time natural language processing in emergency dispatch systems. Master's thesis, Western Norway University of Applied Sciences (HVL), 2023. Master's Thesis.

[5] Cybersecurity and Infrastructure Security Agency (CISA). Next generation 911 systems architecture framework. Technical report, 2023. Accessed: 2025-11-11.

[6] Apache Software Foundation. Apache kafka streams documentation, 2024. Accessed: 2025-11-11.

[7] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2023.

[8] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. In *Proceedings of the NeurIPS EMC2 Workshop*, 2019.

[9] Cybersecurity and Infrastructure Security Agency (CISA). Emergency services sector-specific plan: An annex to the national infrastructure protection plan. Technical report, U.S. Department of Homeland Security, 2021. Accessed: 2025-11-11.

[10] Linh Nguyen, Michael Roberts, and Wen Zhao. Integrating iot data streams into emergency response systems: A scalability perspective. ResearchGate preprint, 2022. Accessed: 2025-11-11.

[11] Mohammad Babaeian, Sina Rahmani, and Sebastian Möller. Improving automatic speech recognition for aviation communications using domain-specific fine-tuning. *Journal of the Acoustical Society of America*, 152(2):821–832, 2022.

[12] Sankaran Panchapagesan, Yongqiang Kim, and Samuel Thomas. Domain adaptation techniques for end-to-end asr in specialized communication environments. *arXiv preprint arXiv:2302.10456*, 2023.

[13] Ajay Kumar, Priya Singh, and Rohit Jain. A comprehensive survey on named entity recognition in clinical texts. *Journal of Biomedical Informatics*, 130:104082, 2022.

[14] Chen Li, Rui Zhao, and Wei Xu. Recent advances in named entity recognition: A survey. *ACM Computing Surveys*, 55(9):1–39, 2023.

[15] Aisha Patel, Deepak Krishnan, and Julia Torres. Entity extraction for emergency medical service records using domain-adapted transformers. *IEEE Journal of Biomedical and Health Informatics*, 28(1):88–99, 2024.

[16] Firoj Alam, Muhammad Imran, and Ferda Ofli. Fine-tuned language models for disaster response: Named entity recognition and classification on crisis data. *arXiv preprint arXiv:2104.08726*, 2021.